

Hisan Labs

DevOps Engineer Interview Preparation Guide

Prepared for: Sanket Ajay Chopade

Role: DevOps Engineer Intern

Date: June 2026

- About the Company & Role Fit
- CI/CD & Jenkins
- Kubernetes & Docker
- Terraform & Infrastructure as Code
- GitOps & Argo CD
- AWS & Cloud Networking
- Monitoring & SRE
- Security & DevSecOps
- Linux & Scripting
- Behavioural Questions (STAR format)

SECTION 1: About Hisan Labs & Role Fit

Q1.

Tell us about Hisan Labs and why you want to work here.

■ What Hisan Labs Does

- Hisan Labs is a cloud consulting and automation company based in India.
- They specialise in AWS cloud migration, GitOps, CI/CD, Security & Compliance, and Site Reliability Engineering (SRE).
- They help startups and mid-sized businesses migrate to AWS using proven strategies with a strong focus on DevOps best practices.

■ Why I Want to Join

- Their core expertise — Kubernetes, Terraform, GitOps, CI/CD — aligns directly with what I worked on during my internship.
- I want to deepen my cloud-native skills in a team that treats DevOps as a product, not an afterthought.
- The SRE and security focus (SOC2, HIPAA, PCI/DSS) gives me exposure to compliance-driven infrastructure, which is a skill gap I want to close.

■ *Tip: Be specific. Mention their GitOps and AWS migration focus, not just 'I like cloud work.'*

Q2.

How does your internship experience match what Hisan Labs does?

■ Direct Overlap

- I built and maintained Jenkins CI/CD pipelines — matching their Release Engineering service.
- I provisioned AWS infrastructure (EKS, VPC, RDS, S3) using Terraform modules — matching their Cloud Migration service.
- I implemented GitOps with Argo CD for automated Kubernetes deployments — directly matching their GitOps service.
- I integrated SonarQube, Trivy, and OWASP into pipelines — matching their Security & Compliance focus.

■ Monitoring (SRE Alignment)

- Set up Prometheus, Grafana, Datadog, and CloudWatch dashboards for proactive incident detection.
- This mirrors their SRE service around SLOs, visibility, and meaningful alerting.

■ *Tip: Use their exact service names (GitOps, Release Engineering, SRE) in your answer to show you read their site.*

SECTION 2: CI/CD & Jenkins

Q3.

Explain how you designed a Jenkins pipeline. Walk us through the stages.

■ Pipeline I Built at Hisan Labs

- Stage 1 — Source: GitHub webhook triggers the pipeline on every push or PR merge.
- Stage 2 — Build: Maven/Docker builds the application and creates a container image.
- Stage 3 — Code Quality: SonarQube scans the code; the build fails if quality gates are not met.
- Stage 4 — Security Scan: Trivy scans the Docker image for known CVEs; OWASP checks dependencies.
- Stage 5 — Push: The verified image is pushed to Amazon ECR with a version tag.
- Stage 6 — Deploy: Kubernetes manifests (or Helm charts) are applied to the target environment.
- Stage 7 — Notify: Slack or email notification sent on success or failure.

■ Key Design Decisions

- Quality and security gates run BEFORE the image is pushed — fail fast, catch issues early.
- Used Jenkins shared libraries to avoid duplicating pipeline code across 3 pipelines.
- Credentials stored in Jenkins Credentials Manager, never hardcoded.

■ *Tip: Draw this pipeline mentally before the interview. Being able to narrate it step-by-step shows real ownership.*

Q4.

What is the difference between a Declarative and Scripted Jenkins pipeline?

■ Declarative Pipeline

- Uses a structured `pipeline {}` block with predefined sections: agent, stages, post.
- Easier to read, write, and validate — recommended for most use cases.
- Syntax errors are caught at parse time before execution begins.

■ Scripted Pipeline

- Written in full Groovy code — gives you maximum flexibility.
- Harder to maintain, fewer guardrails, more prone to errors.
- Used when you need complex logic that Declarative can't handle.

■ What I Used

- I used Declarative pipelines at Hisan Labs for their readability and ease of maintenance in a team setting.

SECTION 3: Kubernetes & Docker

Q5.

What Kubernetes objects did you use and why?

■ Objects Used at Hisan Labs

- Deployment — manages stateless app replicas and handles rolling updates automatically.
- Service — exposes pods internally (ClusterIP) or externally (LoadBalancer/NodePort).
- Ingress — routes external HTTP/HTTPS traffic to correct services using path/host rules.
- ConfigMap — stores non-sensitive configuration data (env variables, config files).
- Secret — stores sensitive data like passwords and API keys in base64-encoded form.
- HorizontalPodAutoscaler (HPA) — automatically scales pod count based on CPU/memory metrics.
- StatefulSet (Three-Tier Project) — used for the database layer where stable identity and persistent storage were needed.

■ *Tip: Know the difference between Deployment (stateless) and StatefulSet (stateful). It's a common follow-up question.*

Q6.

What is a multi-stage Dockerfile and why did you use Distroless images?

■ Multi-Stage Build

- A Dockerfile that uses multiple FROM stages — one for building the app, another for the final runtime image.
- Only the compiled binary or artifact is copied to the final stage; build tools are left behind.
- Result: significantly smaller image size and reduced attack surface.

■ Why Distroless?

- Distroless images (by Google) contain only the runtime (e.g. Go binary) — no shell, no package manager.
- Attackers cannot exec into the container and run shell commands — critical for production security.
- Used this in the Go GitOps pipeline project to minimise the container attack surface.

■ *Tip: Bring up a concrete size comparison if you can — e.g., 'Our image went from ~800MB to ~20MB using this approach.'*

Q7.

What happens when a pod crashes in Kubernetes? How does it recover?

■ Crash Recovery Flow

- The container exits — Kubernetes detects the failure via the kubelet on that node.
- Kubernetes restarts the container based on the pod's restartPolicy (Always by default for Deployments).
- Each restart has an exponential backoff delay: 10s, 20s, 40s... up to 5 minutes.
- If the pod keeps failing, it enters CrashLoopBackOff state — visible in ``kubectl get pods``.

■ Prevention & Debugging

- Set liveness probes to detect deadlocked containers and restart them automatically.
- Set readiness probes to ensure traffic is only sent to healthy pods.
- Debug with: ``kubectl logs``, ``kubectl describe pod``, ``kubectl events``.

SECTION 4: Terraform & Infrastructure as Code

Q8.

What is Terraform remote state and why is it important in a team?

■ What It Is

- Terraform state tracks what real infrastructure exists and maps it to your config files.
- By default, state is stored locally in terraform.tfstate — this breaks in team environments.
- Remote state stores the state file in a shared backend (e.g. AWS S3) so everyone reads the same truth.

■ Why It Matters

- State locking (via DynamoDB) prevents two people from running `terraform apply` at the same time and corrupting state.
- At Hisan Labs, I configured S3-backed remote state with DynamoDB locking for the 4-member team.
- It gave us safe, concurrent infrastructure deployments with zero state conflicts.

■ *Tip: Always mention state locking — remote state without locking is still risky.*

Q9.

What are Terraform modules and why did you use them?

■ What Modules Are

- Modules are reusable groups of Terraform resources — like functions in programming.
- You write the resource config once in a module and call it with different inputs for each environment.

■ How I Used Them

- Created reusable modules for VPC, Security Groups, EKS, RDS, and S3 at Hisan Labs.
- Calling the same module with different variable values (dev/staging/prod) ensured environment consistency.
- Reduced code duplication and made infrastructure changes predictable and testable.

SECTION 5: GitOps & Argo CD

Q10.

What is GitOps and how did you implement it?

■ GitOps Principle

- GitOps = Git is the single source of truth for your infrastructure and application state.
- All changes go through Git (pull requests). No manual kubectl or direct cluster changes.
- An agent (Argo CD) continuously watches the Git repo and syncs the cluster to match it.

■ My Implementation

- Configured Argo CD to watch a Helm chart repo for the production EKS cluster.
- Every merge to main triggered Argo CD to automatically reconcile the cluster state.
- Drift detection: if someone manually changed a Kubernetes object, Argo CD detected it and reverted it.
- Every deployment is auditable — just look at the Git history.

■ *Tip: Hisan Labs has GitOps as a core service offering. Be ready to go deep on this topic.*

SECTION 6: AWS & Cloud

Q11.

Explain how you set up networking for a production application on AWS.

■ VPC Architecture I Provisioned

- Created a VPC with public and private subnets across two Availability Zones for high availability.
- Public subnets: holds the Application Load Balancer (ALB) and NAT Gateway.
- Private subnets: holds EKS worker nodes, RDS database — never directly exposed to the internet.
- Route tables configured so private subnet traffic to the internet goes through the NAT Gateway.
- Security Groups: ALB allows port 80/443 from the internet; nodes only accept traffic from the ALB.

■ *Tip: Draw this architecture in your head as a diagram. Interviewers love when you can describe traffic flow end-to-end.*

Q12.

How did you manage IAM access at Hisan Labs?

■ Approach

- Followed the principle of least privilege — every IAM entity only has the permissions it needs, nothing more.
- Created IAM roles for EC2 instances and EKS node groups instead of embedding credentials in code.
- Created separate IAM users for team members with different permission levels (dev vs admin).
- Used IAM policies attached to roles, never to users directly, for easier management.
- Enabled CloudTrail to audit all API calls for security and compliance.

SECTION 7: Monitoring & SRE

Q13.

How did you set up monitoring on EKS? What tools did you use?

■ Stack Used

- Prometheus — deployed via Helm chart to scrape metrics from EKS nodes and application pods.
- Grafana — connected to Prometheus for dashboards showing CPU, memory, pod restarts, and request rates.
- Datadog — used for APM (Application Performance Monitoring) and log aggregation across services.
- AWS CloudWatch — monitored AWS-level resources: RDS, ALB, EKS control plane, and Lambda.

■ Alerts

- Configured Prometheus alerting rules for pod crash loops, high CPU (>85%), and memory pressure.
- Alerts routed to Slack channels for the on-call engineer to action.

■ *Tip: Hisan Labs mentions SLOs and meaningful alerting in their SRE service. Knowing what triggers alerts (and why) matters.*

SECTION 8: Security & DevSecOps

Q14.

What security tools did you integrate into your CI/CD pipeline?

■ Tools and Purpose

- SonarQube — Static code analysis. Finds bugs, code smells, and security hotspots in source code BEFORE build.
- Trivy — Container image scanner. Detects known CVEs in OS packages and application dependencies in Docker images.
- OWASP Dependency-Check — Scans project dependencies for publicly known vulnerabilities (CVE database).

■ Enforcement

- All three tools were placed as quality gates — the pipeline failed if any critical issues were detected.
- This means no insecure code or vulnerable image ever reached staging or production.

■ *Tip: Hisan Labs offers Security & Compliance as a service. Show that you treat security as non-negotiable, not optional.*

SECTION 9: Linux & Scripting

Q15.

What Bash or Python scripts did you write to automate operations?

■ Examples from My Internship

- Log rotation script — archived old logs, compressed them, and deleted files older than 30 days.
- Environment setup script — provisioned a fresh server with all required packages, users, and configs in one run.
- CI/CD automation script — automated Docker image tagging with Git commit SHA and pushed to ECR.
- Health check script — pinged application endpoints and sent a Slack alert if response time exceeded threshold.

■ Languages Used

- Bash for system-level tasks (file management, process control, environment variables).
- Python for tasks needing API calls, JSON parsing, or more complex logic.

■ Tip: Pick ONE script and explain it line by line if asked. Depth beats breadth here.

SECTION 10: Behavioural Questions

Q16.

Tell me about a time you faced a production issue. How did you handle it?

■ Situation

- During a deployment at Hisan Labs, a Kubernetes pod kept entering CrashLoopBackOff after a release.

■ Action

- Ran `kubectl describe pod`` and `kubectl logs`` to identify the root cause.
- Found a missing environment variable — a ConfigMap key was renamed in the new release but not updated in the Deployment spec.
- Applied a hotfix to the Deployment manifest and the pod recovered immediately.
- Added a post-deploy smoke test to the Jenkins pipeline to catch this class of error in future.

■ Result

- Downtime was under 10 minutes. The pipeline check now prevents this from happening again.

■ Tip: Use the STAR format: Situation, Task, Action, Result. Keep it concise — under 2 minutes when spoken.

Q17.

How do you handle working in a small team where you own multiple responsibilities?

■ My Approach

- At Hisan Labs I was part of a 4-member DevOps team — everyone had to be a generalist.
- I prioritised tasks by impact: production issues first, feature work second, documentation third.
- Used clear Git branching and PR reviews to avoid stepping on each other's infrastructure changes.
- Communicated blockers early rather than silently struggling — speed of communication is key in small teams.

Q18.

Where do you see yourself in 2-3 years in the DevOps/cloud space?

■ Honest, Grounded Answer

- In the next 1-2 years: become proficient in platform engineering — building internal developer platforms that abstract infrastructure complexity for development teams.
- Earn the AWS Solutions Architect Associate certification to solidify cloud architecture fundamentals.
- In 2-3 years: contribute to SRE practices — owning SLOs, error budgets, and reliability engineering at scale.
- Long-term: work with distributed teams on complex cloud-native systems, possibly internationally.

■ *Tip: Show ambition, but tie it to skills you're actively building. Don't say 'I want to be a manager' unless you mean it.*

Q19.

Why should we hire you as an intern over someone with more experience?

■ Honest Value Proposition

- I've already done the job — not in theory, but in a real production environment for 9 months.
- I maintained Jenkins pipelines, provisioned AWS infrastructure with Terraform, and ran GitOps with Argo CD — exactly what Hisan Labs does for clients.
- I'm not starting from zero. I'll need less ramp-up time and can contribute to real projects from week one.
- I'm actively learning (pursuing AWS SAA-C03) and I approach gaps as problems to solve, not blockers.

■ *Tip: Don't be defensive. Lead with what you bring, not with apologies for what you lack.*

This guide was tailored to Hisan Labs' service areas (AWS Migration, GitOps, CI/CD, SRE, Security) and Sanket Chopade's internship experience. Answers are written in your voice — practise them out loud before the interview.